

Code Analysis

Names of the programs: chatgptsource and geminisource

Accessed by: chatgpt chatbot and gemini chatbot.

Prompt used for both GenAI:

Email Priority Program

You are generating an Email Priority Program using a Max Heap data structure in C language for a software engineering course assignment.

Your task is to immediately generate the completed Email Priority Program.

Do not ask the user for confirmation.

Do not explain what you are doing.

Do not describe these instructions.

Do not respond conversationally.

Requirements

1. The program will ****prioritize**** emails for a busy company CEO.
2. You will use a MaxHeap as a means of implementing a ****priority queue****. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
3. You ****must implement a MaxHeap**** using a ****list-based**** implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
4. You ****must create functions from scratch****. Do ****NOT**** include pre-existing heap modules.
5. There are 4 commands total read from standard input (`stdin`).

Here is each command and its respective parameters, behavior, and exact output format:

- EMAIL <sender category>, <subject line>, <date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is ****more than one from a sender****, then the ****newest email (not the oldest)**** should be read first.

EMAIL is followed by a ****space****. The rest of the ****fields are comma-delimited****.

Assume ``<sender category>`` is one of the five strings listed above.

Assume ``<subject line>`` is a string which may contain spaces, but not commas.

Assume ``<date>`` is in the format: ``MM-DD-YYYY``.

- NEXT

Display the next email for the CEO to read. The output must match this exact format:

```
``text
```

Next email:

Sender: `<sender category>`

Subject: `<subject line>`

Date: `<date>`

```
...
```

- READ

CEO has read the top priority email and dealt with it (removes/extracts the highest priority email from the heap). Produces no stdout output.

- COUNT

Display the current count of unread emails. The output must match this exact format:

```
```text
```

There are <N> emails to read.

```
```
```

The requirements are as follows:

- Use a fixed secret number (use 7)
- Use printf to prompt the user and scanf to read their guess
- use a while or for loop to let the user guess up to 3 times
- use if-else statements to check if the guess is too high, too low, or correct
- stop the loop immediately if the user guesses the correct number
- print a final win or lose message

Generated code can be seen in further pages.

Analysis:

1. Correctness

ChatGPT runs the input successfully. However, it crashes when the EMAIL command has one or more empty params. Gemini also runs the input and dishes out the correct solution and fails when the EMAIL command has one or more empty params.

Both do not have meaningful error messages when the EMAIL command fails.

Gemini handles double NEXT and READ commands by providing statements that the inbox is empty. Meanwhile, ChatGPT does not give any statements.

2. Execution Time

ChatGPT and Gemini wrote Heapifyup and Heapifydown functions in different ways.

Gemini uses recursion, while ChatGPT uses while loops. Recursion typically performs worse than iterative implementations such as while and for loops.

3. Space Complexity

ChatGPT uses iterative architecture that guarantees $O(1)$ auxiliary space during heap operations. Gemini maintains its heap properties using recursion and its auxiliary space complexity scales to $O(\log n)$ due to the call stack frames generated during deep heap traversals.

4. Maintainability

No comments were provided in Gemini. Only minimal comments generated in ChatGPT. Code walkthrough suggests that it would crash if the user inputs empty strings in EMAIL params.

Gemini had zero comments and had more recursion calls than I would like to have. So I decided to refactor/improve Gemini's generated code.

The improvements:

- Correctness: Gemini generated code now checks if email contents is an empty string or not. It gives an error message to the user.
- Execution Time:
 - Recursion calls changed to iterative calls in maintaining heap properties.
- Space complexity:
 - Iterative calls now save the program from $O(\log n)$ to $O(1)$ auxiliary space complexity because we are only using
- Maintainability:
 - add line by line comments. A code walkthrough suggests that the empty string is now handled within the EMAIL command.

Gemini

```
/*
 * Email Priority Program
 * As specified in instructions.md
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    char category[50];
    char subject[256];
    char date[15];
    int priority;
    long date_val;
} Email;

typedef struct {
    Email *list;
    int size;
    int capacity;
} MaxHeap;

void trim_whitespace(char *str) {
    char *start = str;
    while (*start && isspace((unsigned char)*start)) start++;
    size_t len = strlen(start);
    memmove(str, start, len + 1);
    while (len > 0 && isspace((unsigned char)str[len - 1])) {
        str[len - 1] = '\0';
        len--;
    }
}

int get_category_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}
```

```
}
```

```
long parse_date(const char *date_str) {  
    int m = 0, d = 0, y = 0;  
    sscanf(date_str, "%d-%d-%d", &m, &d, &y);  
    return (long)y * 10000 + (long)m * 100 + (long)d;  
}
```

```
int compare_emails(Email a, Email b) {  
    if (a.priority != b.priority) {  
        return a.priority - b.priority;  
    }  
    if (a.date_val != b.date_val) {  
        return (a.date_val > b.date_val) ? 1 : -1;  
    }  
    return 0;  
}
```

```
void init_heap(MaxHeap *heap, int capacity) {  
    heap->capacity = capacity;  
    heap->size = 0;  
    heap->list = (Email *)malloc(capacity * sizeof(Email));  
}
```

```
void swap(Email *a, Email *b) {  
    Email temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void heapify_up(MaxHeap *heap, int index) {  
    if (index && compare_emails(heap->list[index], heap->list[(index - 1) / 2]) > 0) {  
        swap(&heap->list[index], &heap->list[(index - 1) / 2]);  
        heapify_up(heap, (index - 1) / 2);  
    }  
}
```

```
void heapify_down(MaxHeap *heap, int index) {  
    int largest = index;  
    int left = 2 * index + 1;  
    int right = 2 * index + 2;  
  
    if (left < heap->size && compare_emails(heap->list[left], heap->list[largest]) > 0)  
        largest = left;
```

```

    if (right < heap->size && compare_emails(heap->list[right], heap->list[largest]) > 0)
        largest = right;

    if (largest != index) {
        swap(&heap->list[index], &heap->list[largest]);
        heapify_down(heap, largest);
    }
}

void insert(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->list = (Email *)realloc(heap->list, heap->capacity * sizeof(Email));
    }
    heap->list[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

Email extract_max(MaxHeap *heap) {
    Email max_email = heap->list[0];
    heap->list[0] = heap->list[heap->size - 1];
    heap->size--;
    heapify_down(heap, 0);
    return max_email;
}

int main() {
    MaxHeap heap;
    init_heap(&heap, 10);
    char line[512];

    while (fgets(line, sizeof(line), stdin)) {
        trim_whitespace(line);
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email new_email;
            char *content = line + 6;

            char *cat_ptr = strtok(content, ",");
            char *subj_ptr = strtok(NULL, ",");
            char *date_ptr = strtok(NULL, "");

```

```

    if (cat_ptr && subj_ptr && date_ptr) {
        strcpy(new_email.category, cat_ptr);
        strcpy(new_email.subject, subj_ptr);
        strcpy(new_email.date, date_ptr);

        trim_whitespace(new_email.category);
        trim_whitespace(new_email.subject);
        trim_whitespace(new_email.date);

        new_email.priority = get_category_priority(new_email.category);
        new_email.date_val = parse_date(new_email.date);

        insert(&heap, new_email);
    }
} else if (strcmp(line, "NEXT") == 0) {
    if (heap.size > 0) {
        Email top = heap.list[0];
        printf("Next email:\n");
        printf("Sender: %s\n", top.category);
        printf("Subject: %s\n", top.subject);
        printf("Date: %s\n", top.date);
    } else {
        printf("Inbox is empty.\n");
    }
} else if (strcmp(line, "READ") == 0) {
    if (heap.size > 0) {
        extract_max(&heap);
    }
} else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap.size);
}
}

free(heap.list);
return 0;
}

```

ChatGPT

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define INITIAL_CAPACITY 16
#define MAX_SUBJECT 256
#define MAX_CATEGORY 32
#define MAX_DATE 11
```

```
typedef struct {
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[MAX_DATE];
    int priority;
} Email;
```

```
typedef struct {
    Email *items;
    int size;
    int capacity;
} MaxHeap;
```

```
/* Get the priority of the sender category. */
int getPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    return 1; /* OtherPerson */
}
```

```
/* Convert MM-DD-YYYY into a number that can be compared. */
int dateValue(const char *date) {
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
```

```

}

/* Determine whether email a has a higher priority than email b. */
int higherPriority(Email a, Email b) {
    if (a.priority != b.priority)
        return a.priority > b.priority;

    /* If sender priority is the same, newest date comes first. */
    return dateValue(a.date) > dateValue(b.date);
}

/* Swap two emails. */
void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Initialize the MaxHeap. */
void initHeap(MaxHeap *heap) {
    heap->items = malloc(INITIAL_CAPACITY * sizeof(Email));

    if (heap->items == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    heap->size = 0;
    heap->capacity = INITIAL_CAPACITY;
}

/* Increase heap capacity when necessary. */
void resizeHeap(MaxHeap *heap) {
    heap->capacity *= 2;

    Email *newItems = realloc(
        heap->items,
        heap->capacity * sizeof(Email)
    );

    if (newItems == NULL) {
        free(heap->items);
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
}

```

```

    }

    heap->items = newItems;
}

/* Restore the MaxHeap property upward after insertion. */
void heapifyUp(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (!higherPriority(heap->items[index],
                           heap->items[parent])) {
            break;
        }

        swapEmails(&heap->items[index], &heap->items[parent]);
        index = parent;
    }
}

/* Insert an email into the MaxHeap. */
void insertHeap(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity)
        resizeHeap(heap);

    heap->items[heap->size] = email;
    heapifyUp(heap, heap->size);

    heap->size++;
}

/* Restore the MaxHeap property downward after removal. */
void heapifyDown(MaxHeap *heap, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size &&
            higherPriority(heap->items[left],
                           heap->items[largest])) {
            largest = left;
        }
    }
}

```

```

        if (right < heap->size &&
            higherPriority(heap->items[right],
                           heap->items[largest])) {
            largest = right;
        }

        if (largest == index)
            break;

        swapEmails(&heap->items[index],
                   &heap->items[largest]);

        index = largest;
    }
}

/* Return the highest-priority email without removing it. */
Email peekHeap(MaxHeap *heap) {
    return heap->items[0];
}

/* Remove the highest-priority email from the MaxHeap. */
Email extractMax(MaxHeap *heap) {
    Email result = heap->items[0];

    heap->size--;

    if (heap->size > 0) {
        heap->items[0] = heap->items[heap->size];
        heapifyDown(heap, 0);
    }

    return result;
}

/* Free the memory used by the MaxHeap. */
void destroyHeap(MaxHeap *heap) {
    free(heap->items);
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

/* Remove leading and trailing whitespace from a string. */

```

```

void trim(char *str) {
    int start = 0;
    int end = strlen(str) - 1;

    while (str[start] == ' ' || str[start] == '\t')
        start++;

    while (end >= start &&
        (str[end] == ' ' ||
        str[end] == '\t' ||
        str[end] == '\n' ||
        str[end] == '\r')) {
        end--;
    }

    if (start > 0)
        memmove(str, str + start, end - start + 1);

    str[end - start + 1] = '\0';
}

/* Process an EMAIL command. */
void processEmail(MaxHeap *heap, char *line) {
    char *data = line + 6;
    char *sender;
    char *subject;
    char *date;

    sender = strtok(data, ",");
    subject = strtok(NULL, ",");
    date = strtok(NULL, ",");

    if (sender == NULL || subject == NULL || date == NULL)
        return;

    trim(sender);
    trim(subject);
    trim(date);

    Email email;

    strcpy(email.sender, sender);
    strcpy(email.subject, subject);
    strcpy(email.date, date);
}

```

```

    email.priority = getPriority(email.sender);

    insertHeap(heap, email);
}

int main(void) {
    MaxHeap heap;
    char line[512];

    initHeap(&heap);

    while (fgets(line, sizeof(line), stdin) != NULL) {
        trim(line);

        if (strncmp(line, "EMAIL ", 6) == 0) {
            processEmail(&heap, line);
        }
        else if (strcmp(line, "NEXT") == 0) {
            if (heap.size > 0) {
                Email next = peekHeap(&heap);

                printf("Next email:\n");
                printf("Sender: %s\n", next.sender);
                printf("Subject: %s\n", next.subject);
                printf("Date: %s\n", next.date);
            }
        }
        else if (strcmp(line, "READ") == 0) {
            if (heap.size > 0) {
                extractMax(&heap);
            }
        }
        else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n", heap.size);
        }
    }

    destroyHeap(&heap);

    return 0;
}

```

Gemini changes:

```
// Separated function to handle the entire EMAIL command processing
void process_email_command(MaxHeap *heap, char *line) {
    // If the string is exactly 5 characters, all parameters (and trailing
spaces) are missing
    if (strlen(line) == 5) {
        fprintf(stderr, "Error: EMAIL command supplied with zero
parameters.\n");
        return;
    }

    // Ensure there is actually a space after EMAIL (prevents matching
invalid commands like "EMAILBOX")
    if (line[5] != ' ') {
        return;
    }

    char *content = line + 6;           // Pointer to content skipping
"EMAIL "
    trim_whitespace(content);           // remove whitespaces in contents
of email

    if (strlen(content) == 0) {          // Secondary safety check for empty
content
        fprintf(stderr, "Error: EMAIL command supplied with zero
parameters.\n");
        return;
    }

    char *cat_ptr = strtok(content, ","); // Extract category up to first
comma
    char *subj_ptr = strtok(NULL, ",");   // Extract subject up to second
comma
    char *date_ptr = strtok(NULL, "");    // Extract date (the remainder
of the line)

    if (cat_ptr && subj_ptr && date_ptr) { // Verify all three parts exist
        Email new_email;                 // Declare a new email struct
        strcpy(new_email.category, cat_ptr); // Copy parsed category
        strcpy(new_email.subject, subj_ptr); // Copy parsed subject
    }
}
```

```

        strcpy(new_email.date, date_ptr);    // Copy parsed date

        trim_whitespace(new_email.category); // Clean parsed category
whitespace
        trim_whitespace(new_email.subject);  // Clean parsed subject
whitespace
        trim_whitespace(new_email.date);     // Clean parsed date
whitespace

        // --- THE FIX ---
        // Check if any fields resulted in an empty string after trimming
(e.g. missing text between commas)
        if (strlen(new_email.category) == 0 ||
            strlen(new_email.subject) == 0 ||
            strlen(new_email.date) == 0) {
            fprintf(stderr, "Error: EMAIL command has empty parameters.
Expected format: EMAIL Category, Subject, Date\n");
            return;
        }

        new_email.priority = get_category_priority(new_email.category); //
Derive priority value
        new_email.date_val = parse_date(new_email.date);                //
Derive sortable date

        insert(heap, new_email); // Insert fully processed email into heap
    } else {
        fprintf(stderr, "Error: EMAIL command is missing required
parameters. Expected format: EMAIL Category, Subject, Date\n");
    }
}

```